Lab2 - Simple Design

This lab session provides you with your initial practical experience in verifying a specific design. In this lab, the device under test (DUT) will be treated as a black box, meaning our understanding of it is limited to what is provided in the specification document. The student will choose the most suitable verification strategy based on the design.

1. Get Ready for this Lab

In this lab you will achieve your first verification of a simple design. To start, in the web of the course you will find a compressed package alu_4bit.tgz with the following files:

```
$ tar xvfz alu_4bit.tgz
alu_4bit_tb.sv
modelsim.ini
work/
```

The work/ directory contains a QuestaSim library with 6 RTL designs already compiled with vlog. The design modules are named alu_4bit_v0, alu_4bit_v1, ..., and alu_4bit_v5. All these modules represent alternative RTL implementations of the design specification. As said, all these designs are provided already compiled, ensuring that the Verilog code remains hidden from students. Only alu_4bit_v0 is provided as a correct RTL design. The other 5 designs are supposed to implement the same specification, but none, some, or all may be incorrect and have bug/s. It is your role to determine if each design is correct (PASS) or incorrect (FAIL).

The compressed package also contains an empty alu_4bit_tb.sv file for you to write a testbench. The goal of this session is to read carefully the specifications (in the next section, it is supposed to be the same for the design and the verification engineer), create a verification plan, and write a testbench which runs one or several tests to determine if the designs contain error/s.

2. Design Specification

2.1. Overview

This specification describes the behavior of a 4-bit Arithmetic Logic Unit (ALU) to be implemented in Verilog for the Simple Design laboratory. The ALU performs basic arithmetic and logic operations on two 4-bit operands A and B according to a 3-bit operation selector op.

Both the *design team* (responsible for RTL) and the *verification team* (responsible for testbench and checks) shall use this specification as their only reference.

2.2. Functional Description

The ALU computes a 4-bit result Y and a carry/borrow flag carry depending on the operation selected. Operations are purely combinational; there are no clocks or resets in this version.

Opcode (op)	Operation	Description	Example (A=3, B=2)	Y	carry
000	ADD	Y = A + B	3 + 2	0101	Carry = 0
001	SUB	Y = A - B (two's complement)	3 - 2	0001	Carry = 0
010	AND	Bitwise AND	3 & 2	0010	0
011	OR	Bitwise OR	3 | 2	0011	0
100	XOR	Bitwise XOR	3 ^ 2	0001	0
101	NOT A	Bitwise negation of A	~3	1100	0
110	PASS A	Output A	3	0011	0
111	PASS B	Output B	2	0010	0

2.3. Interface Definition

Following table describes the specification for the input/output signals.

Signal	Dir	Width	Description
A	Input	4	Operand A
B	Input	4	Operand B
op	Input	3	Operation selector
Y	Output	4	ALU result
carry	Output	1	Carry/borrow out (only valid for add/sub)

2.4. Behavioral Rules

Following are behavioral rules for specific operations with special conditions:

1) Addition:
    a. Perform unsigned addition of A and B.
    b. Y = lower 4 bits of the sum.
    c. carry = MSB of the 5-bit result.
2) Subtraction:
    a. Perform unsigned subtraction (A - B).
    b. If A < B, result wraps around (mod 16).
    c. carry = 1 if borrow occurred (optional interpretation, see note below). (In this lab, carry=MSB of A - B result, consistent with addition logic.)
3) Logical operations:
    a. carry must be set to 0.
4) Invalid opcode handling:
    a. Drive Y = 0000 and carry = 0.

3. How to Proceed

Here are the steps you need to take to complete this lab effectively:

A. Prepare a verification plan

Take time to thoroughly review the design specifications and clearly define the requirements and features that need verification. This process will guide you in preparing a list of tests for your testbench and help you determine the most effective testing strategy.

YOU ARE NOT REQUIRED TO IMPLEMENT A FULL UVM-STYLE TESTBENCH FOR THIS LAB SESSION, AS DISCUSSED IN THE THEORY CLASS. THE DECISION IS YOURS; A SIMPLER APPROACH IS ALSO ACCEPTABLE.

B. Write your testbench

After completing your testbench as alu_4bit_tb.sv (recall that each design module can be instantiated by its module name), you can load and compile your testbench and design as:

```
QuestaSim> vmap work ./work
QuestaSim> vdir
# Library vendor : Model Technology
# Maximum unnamed designs : 3
# MODULE alu_4bit_v0
# MODULE alu_4bit_v1
# MODULE alu_4bit_v2
# MODULE alu_4bit_v3
# MODULE alu_4bit_v4
# MODULE alu_4bit_v5
QuestaSim> vlog +acc alu_4bit_tb.sv
QuestaSim> vsim work.alu_4bit_tb
QuestaSim> run -all
```

If you prefer, you can perform the build process from command line using a Makefile.

C. Try your testbench with every provided design (just change the module name in the instantiation)

4. What to Turn In

After the 2h lab session please send me through ElRaco Pràctiques your current draft of the verification plan and/or your testbench.

Before the final deadline for the session please provide a hard copy (IN A SINGLE PDF DOCUMENT, DO NOT SEND ME SEPARATE FILES, PLEASE) of each of the following items:

1. Please indicate how many hours you spent on this lab. This will not affect your grade, but will be helpful for calibrating the workload for the future.
2. (2 pts) Your verification plan
3. (4 pts) Your testbench alu_4bit_tb.sv code. How many different tests did you write? (explain)
4. (2 pts) Waveform showing several tests for the ADD operations
5. (2 pts) A table indicating for each of the designs 1-5, if they PASS/FAIL, and the reason in case of FAIL (aka. which of your test/s failed)